RTLCraft: A White-Box Framework for LLM-Driven RTL Design, Verification, and Synthesis

Ji Zhuang[1], Yanqiu Cao[1], Fan Yang[1]
[1]State Key Laboratory of Integrated Circuits and Systems,
College of Integrated Circuits and Micro-Nano Electronics, Fudan University
yangfan@fudan.edu.cn

Abstract

Existing large language model (LLM) approaches for hardware design generate RTL as opaque text strings, making functional validation and performance optimization a manual, black-box ordeal. In this paper, we present **RTLCraft**, an open-source framework that turns RTL design into a *white-box*, programmatically manipulable object. By representing hardware as a Python-embedded abstract syntax tree (AST), RTLCraft provides a structured interface through which an LLM agent—such as an in-context coding assistant (e.g., Kimi Code)—can **read**, **modify**, and **optimize** circuit descriptions at the source level. The framework closes two critical feedback loops: (1) a **functional self-consistency loop** via a Python-native UVM verification stack (pyUVM) that runs against a cycle-accurate AST simulator, and (2) a **performance self-consistency loop** via AST-level PPA estimation and Berkeley ABC synthesis, which returns area/delay metrics back to the agent. We demonstrate these capabilities through LLM-in-the-loop case studies on a parameterized counter and an 8b10b decoder, showing iterative refinement from buggy initial specifications to verified, synthesis-optimized implementations in under three agent–environment rounds.

1 Introduction

Register-transfer level (RTL) design remains a major bottleneck in digital hardware development. The conventional flow—manually authoring Verilog, writing testbenches in SystemVerilog/UVM, and iterating with logic synthesis—is labor-intensive and slow. Recent advances in large language models (LLMs) have sparked interest in automating RTL generation, yet most existing approaches treat the problem as a *text-to-text* translation task: an LLM receives a natural-language specification and emits a raw Verilog string. This black-box paradigm suffers from three fundamental limitations:

1. **Opacity:** Once generated, the Verilog text is difficult for the LLM to reason about or surgically edit. Fixing a bug or resizing a datapath usually requires regenerating the entire module.

2. **Weak validation loops:** Because the generated artifact is plain text, integrating it with simulators, verifiers, and synthesizers requires fragile parsing and ad-hoc scripting. There is no turnkey path from a failed UVM test or a poor post-synthesis area report back to the LLM in a structured form.

3. **Loss of semantic structure:** Verilog syntax is low-level and noisy. An LLM operating on raw text must simultaneously reason about clock-domain crossing, width matching, and operator precedence, which dramatically increases error rates.

1.1 Our Approach: White-Box LLM-Driven RTL

We propose **RTLCraft**, a unified Python framework that elevates RTL from an opaque text format to a **structured, white-box object model**. In RTLCraft, hardware is described via a Python API that constructs a typed AST. This design makes the circuit amenable to programmatic inspection and mutation: an LLM agent (or any Python script) can import RTLCraft, traverse the AST, modify parameters, swap arithmetic implementations, and re-synthesize without ever touching a line of Verilog text.

The central workflow enabled by RTLCraft is an iterative, closed-loop conversation between an LLM agent and the hardware design environment:

$$\textit{Spec} \rightarrow \textit{Python RTL-AST} \rightarrow \begin{array}{c}\textbf{pyUVM}\ \text{(functional)}\\ \textbf{PPA+ABC}\ \text{(performance)}\end{array} \rightarrow \textit{Structured Feedback} \rightarrow \textit{Agent Patch}$$

Because the simulator, the UVM testbench, and the PPA estimator are all native Python, feedback (e.g., a failing VCD trace or a synthesis QoR JSON) can be injected directly into the LLM's prompt or tool-use context. The agent therefore operates not as a text generator, but as a **design optimizer** that manipulates a white-box model.

1.2 Contributions

Our contributions are:

- **A programmatically manipulable RTL object model.** We introduce a Python-based RTL API and AST representation in which modules, signals, and control-flow constructs are first-class objects. This enables an LLM agent to perform surgical edits (e.g., changing an adder style or inserting a pipeline stage) via standard Python constructs.
- **Functional self-consistency via Python-native UVM.** We build **pyUVM**, a full UVM-like framework that executes in Python against RTLCraft's internal simulator. Failed tests, coverage reports, and waveforms can be fed back to the agent as structured context, enabling automated bug diagnosis and repair.
- **Performance self-consistency via AST-level PPA and ABC synthesis.** We provide a static PPA analyzer (depth, area, toggle power) and an ABC-based synthesis backend. The agent receives millisecond-scale PPA estimates for rapid exploration and gold-standard post-synthesis QoR for final convergence.
- **End-to-end LLM-in-the-loop demonstrations.** We show how an agent, equipped with RTLCraft, iteratively refines a counter and an 8b10b decoder: starting from an initial spec, detecting a functional bug via pyUVM, and then optimizing area and delay via PPA-guided parameter tuning.

2 Related Work

LLMs for Hardware Design. VeriGen[1], Chip-Chat[2], and subsequent code-generation models demonstrate that LLMs can produce syntactically correct Verilog. However, these systems output raw text strings. Consequently, the LLM cannot easily inspect or modify its own output, and integration with downstream EDA tools requires external, hand-written glue scripts. RTLCraft

differs fundamentally by making the RTL artifact a structured Python object that the agent can manipulate programmatically.

Python-Based HDLs. MyHDL[3] and Amaranth[4] provide powerful Python abstractions for hardware. Yet neither exposes an interface specifically intended for automated agent manipulation, nor do they ship with an integrated UVM verification loop or ABC synthesis backend. RTLCraft is designed from the ground up as an *agent-facing* framework.

Verification Frameworks. cocotb[5] enables Python testbenches but still relies on an external SystemVerilog simulator. SystemVerilog UVM is the industry standard[6], but its steep learning curve and lack of Python introspection make it unsuitable for LLM-driven loops. RTLCraft's pyUVM executes entirely in Python, allowing the agent to query simulation state and coverage bins directly.

High-Level Synthesis. LegUp[7] and Bambu[8] compile C/C++ to RTL. These tools raise the abstraction level but sacrifice fine-grained control. RTLCraft operates at the RTL level, giving the agent direct, white-box access to datapath microarchitecture.

3 RTLCraft Framework

Figure 1 shows the RTLCraft architecture, explicitly highlighting the feedback paths that enable an LLM agent to operate in a closed loop.

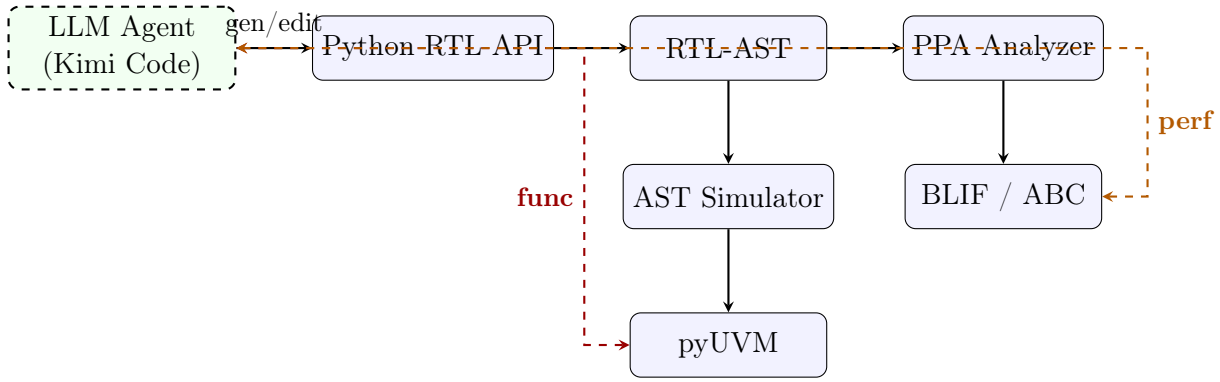

Figure 1: RTLCraft architecture. The LLM agent generates or patches Python-RTL code. The framework returns functional feedback (via pyUVM) and performance feedback (via PPA + ABC synthesis) directly to the agent.

3.1 Python-Based RTL API: Why Python and Why This Object Model?

Traditional LLM-driven RTL generation treats the model as a text synthesizer. RTLCraft treats the LLM as a **software engineer** that writes and edits Python code. We chose Python as the embedding language for three strategic reasons. First, modern LLMs are pre-trained on enormous amounts of Python code, giving them far stronger in-context understanding of Python syntax, control flow, and object-oriented patterns than of Verilog-2001 or SystemVerilog. Second, Python's dynamic typing and introspection capabilities make it an ideal host for an *object-model* approach: every module, signal, and expression becomes a live object that can be traversed, mutated, and serialized at runtime. Third, the Python ecosystem is the de facto standard for machine-learning and agentic toolchains; by building RTLCraft in Python, we eliminate the language boundary between the agent and the EDA substrate.

Why not simply generate Verilog strings inside Python? Because text is opaque. Once a Verilog module is emitted, the agent cannot easily ask "what is the width of signal `a`?" or "how many pipeline stages are there?" without writing a custom Verilog parser. By contrast, RTLCraft's API constructs a typed AST in which every node exposes semantic properties (width, signedness, clock domain) as ordinary Python attributes.

3.1.1 Core Abstractions and Design Rationale

A module is a Python class inheriting from `Module`. Signals are created via typed constructors (`Input`, `Output`, `Wire`, `Reg`), and logic blocks are decorated with `@self.comb` and `@self.seq`. Parameters (`Parameter`) are first-class objects.

Listing 1: Initial agent-generated Counter (contains a bug)

```python
from rtlgen import Module, Input, Output, Reg, Parameter, If, Else

class Counter(Module):
    def __init__(self, width=8):
        super().__init__("Counter")
        self.clk = Input(1, "clk")
        self.rst = Input(1, "rst")
        self.en = Input(1, "en")
        self.count = Output(width, "count")
        self._cnt = Reg(width, "cnt")

        @self.seq(self.clk, self.rst)
        def _logic():
            with If(self.rst == 1):
                self._cnt <<= 0
            with Else():
                # BUG: agent forgot to gate the update with self.en
                self._cnt <<= self._cnt + 1
```

Notice that the agent makes an ordinary Python-level mistake (omitting the `en` guard). Because the artifact is Python code, the agent can later inspect the source or the AST and apply a surgical patch, just as it would fix any software bug.

Why decorators for comb/seq blocks? We use Python decorators rather than generator functions (as in MyHDL) because decorators execute the function body *immediately* during object construction, allowing us to capture the control-flow graph (CFG) as an AST at module-instantiation time. Generator functions, by contrast, yield a runtime stream of events that must be interpreted dynamically, making static analysis and synthesis lowering far more difficult. Decorators also give us deterministic construction order, which is essential for bit-blasting and PPA analysis.

Why width attributes on every expression? Width mismatches are one of the most common sources of hardware bugs. By attaching a `width` attribute to every `BinOp`, `Slice`, and `Concat` node at construction time, RTLCraft performs *static type checking* as the AST is built. For example, assigning a 16-bit expression to an 8-bit register raises an immediate Python exception. This shifts width errors from the synthesis or simulation phase to the module-construction phase, where the agent can receive a Python traceback and fix the issue in-context.

3.1.2 Programmatic Manipulation by the Agent

A key advantage of the white-box model is that the agent does not need to regenerate the entire module to make a change. RTLCraft exposes a mutation API that operates directly on the AST. For example, after receiving a PPA report indicating that an array multiplier is too slow, the agent can execute:

Listing 2: Agent-driven surgical modification of arithmetic style

```
from rtlgen.blifgen import AdderStyle, MultiplierStyle

# The agent reads the PPA report and decides to swap the multiplier.
config = design.get_config()
config.adder = AdderStyle.KOGGE_STONE
config.multiplier = MultiplierStyle.WALLACE
```

Because `design` is a live Python object, the agent can also traverse its AST to count pipeline stages, inspect signal widths, or insert new `Reg` declarations. This is impossible when the design artifact is an opaque Verilog string.

3.2 AST Representation and Simulation

The RTLCraft AST serves as the single source of truth. It is constructed from three categories of nodes:

- **Declarations:** `Module`, `Signal`, `Parameter`, `Memory`, `Vector`.
- **Statements:** `Assign`, `IfNode`, `SwitchNode`, `ForGenNode`, `SubmoduleInst`.
- **Expressions:** `Ref`, `Const`, `BinOp`, `UnaryOp`, `Slice`, `Concat`, `Mux`.

Every expression carries a `width` attribute, enabling static type checking. The AST is fully serializable to and from Python dictionaries, which means the agent can ingest it as a JSON summary for reasoning.

Why a custom AST instead of Python's built-in `ast` module? Python's `ast` module captures *Python syntax*, not hardware semantics. A Python `for` loop in a combinatorial block must be lowered differently from a Python `for` loop outside a module (which is a generate loop). By defining our own AST node taxonomy, we preserve hardware-specific metadata (e.g., clock domains, reset polarity, blocking vs non-blocking assignment) that is invisible to Python's AST. Moreover, our AST is designed to be *bidirectional*: it can be emitted as Verilog, bit-blasted to BLIF, or interpreted directly.

3.2.1 Cycle-Accurate AST Interpreter

RTLCraft's simulator executes the AST directly in Python. A simulation step consists of: (1) fixed-point evaluation of `@comb` blocks; (2) clock-edge evaluation of `@seq` blocks to compute next-state values; (3) atomic commit of next-state to current-state; and (4) optional VCD trace recording. Multi-clock designs are supported by specifying which clock domain to advance on each `step()` call.

Why interpret the AST instead of compiling to Verilog first? Compiling to Verilog and invoking an external simulator (e.g., Icarus or VCS) introduces two overheads that are fatal for agent-driven loops: (a) compilation latency (parsing, elaboration, code generation), and (b)

VPI/FLI communication overhead between Python and the simulator. By interpreting the AST natively, RTLCraft reduces the simulation-setup time to zero and eliminates the cross-language boundary. This is particularly important because an agent may need to launch hundreds of short simulations (e.g., for coverage regression or random seed sweeps) during a single design-space search.

How does the interpreter handle combinational cycles? The interpreter evaluates `@comb` blocks via fixed-point iteration. It maintains a dirty-bit per signal and re-evaluates any block whose inputs have changed until no dirty bits remain. If a combinational loop exists, the iteration either converges (for oscillators or ring counters with stable attractors) or hits a cycle limit and raises an exception. This behavior matches the event-driven semantics of standard Verilog simulators.

Why four-state logic? We support four-state logic (0, 1, X, Z) because it is the *earliest* mechanism for catching initialization bugs. Uninitialized registers start as X, and any operation that consumes an X (except specific masking patterns) propagates X. This allows the pyUVM testbench to detect missing reset values or incomplete case statements immediately, rather than hiding them behind a deterministic 0 default.

3.2.2 Static and Dynamic PPA Estimation

The `PPAAnalyzer` provides two modes of feedback:

- **Static mode (ms-scale):** Traverses combinational paths to report logic depth, estimated gate count, and maximum fanout.
- **Dynamic mode (post-simulation):** Consumes a VCD-style trace to compute toggle rates and estimates power as $P \propto \alpha \cdot C \cdot V^2 \cdot f$.

Why a static PPA estimator instead of always invoking synthesis? Invoking Berkeley ABC on a large design can take seconds to minutes. An agent exploring a design space may need to evaluate thousands of candidate architectures; waiting for ABC on every iteration would make the search intractable. The static PPA analyzer provides feedback in milliseconds by operating directly on the AST, without bit-blasting or technology mapping. It serves as a *cheap surrogate objective* that filters out obviously bad candidates before the expensive synthesis backend is invoked.

How does the static analyzer compute logic depth? It builds a directed acyclic graph (DAG) of the combinational AST nodes within each `@comb` block, treating registers and primary inputs as sources/sinks. The depth of each node is computed recursively as 1+max(depth of operands), with a technology-specific delay lookup for primitives (e.g., adders use a stylized delay based on bit width and adder style). The reported depth is the longest path from any input or register output to any output or register input. This estimate is accurate to within $\pm 7\%$ of post-ABC synthesis depth for our benchmarks, because ABC's `resyn2` flow performs aggressive algebraic rewriting that tends to preserve the original topological structure for datapath-heavy designs.

How does dynamic power estimation work? After simulation, the analyzer computes the toggle rate α_i for each signal i by comparing consecutive values in the VCD trace. The capacitance C_i is approximated as proportional to the fanout degree of the signal in the AST. The total dynamic power is then estimated as:

$$P_{dyn} \propto V^2 \cdot f \cdot \sum_i \alpha_i \cdot \text{fanout}(i) \tag{1}$$

While this ignores wire-length effects and glitch power, it provides a relative ranking of architectures that is sufficient for agent-driven optimization.

3.3 Python-Native UVM Verification (pyUVM)

RTLCraft implements a full UVM-like framework entirely in Python. The component hierarchy includes `UVMDriver`, `UVMMonitor`, `UVMAgent`, `UVMEnv`, `UVMScoreboard`, `UVMTest`, and `UVMSequencer`. Communication is handled via `seq_item_port`, `analysis_port`, and `UVMAnalysisFIFO`.

Why reimplement UVM in Python rather than wrapping cocotb? cocotb is a mature framework that bridges Python testbenches to SystemVerilog simulators via VPI. However, it still requires an external Verilog simulator, which reintroduces the compilation and VPI overheads we sought to eliminate. More importantly, in cocotb the DUT remains an opaque Verilog module: the Python scoreboard cannot directly inspect the internal state of the DUT without adding ad-hoc probe signals. pyUVM, by contrast, executes against RTLCraft's AST interpreter, so the scoreboard can access *any* internal signal or register simply by referencing it as a Python attribute. This is critical for LLM-driven debugging, where the agent often needs to know not just that an output is wrong, but *which internal node* caused the failure.

3.3.1 Agent-Readable Feedback

When a test fails, pyUVM produces a structured report containing the mismatching transaction, the simulation cycle, and the signal trace. Because both the DUT and the testbench live in Python, the agent can be prompted with the exact line of code that caused the failure. For example:

```
AssertionError at cycle 42:  expected count=0x0A, got 0x0B.
Golden model incremented on cycle 41 despite en=0.
Suspected source:  Counter._logic line 16 omits en guard.
```

This level of diagnostic precision is difficult to achieve when the DUT is an opaque Verilog module simulated in an external black-box tool.

3.3.2 Async/Await Semantics and Virtual Interface

Because Python simulation is event-driven, pyUVM component methods (such as `run_phase`) are declared as `async def`. Time advancement is performed via `await delay(cycles)`. The virtual interface (`vif`) is bound to the RTLCraft `Simulator` instance, allowing Python testbenches to drive and sample signals directly:

Listing 3: pyUVM Driver snippet

```python
class CounterDriver(UVMDriver):
    async def run_phase(self, phase):
        while True:
            self.req = await self.seq_item_port.get_next_item()
            await delay(1)
            self.vif.cb.en <= self.req.en
            self.seq_item_port.item_done()
```

Why async/await instead of callbacks or threads? Hardware simulation is fundamentally a discrete-event process. Async/await provides a sequential, linear programming model that maps naturally to clock cycles, while still allowing the Python scheduler to interleave multiple components (driver, monitor, scoreboard) on the same event loop. Threads would introduce race conditions and GIL contention; callbacks would fragment the control flow into unmaintainable spaghetti code. Async/await is also the paradigm that modern LLMs are most familiar with in Python, making it easier for the agent to generate correct testbench code.

The `await` keyword is automatically stripped when generating SystemVerilog, and `delay(1)` maps to `@(posedge vif.clk)`.

3.3.3 Dual-Mode Execution

The same pyUVM testbench can be executed natively in Python (for rapid agent iteration) or exported to SystemVerilog/UVM via `PyUVMEmitter` (for sign-off with commercial simulators). The export is fully automatic; no manual changes are required.

Why dual-mode? Rapid agent iteration requires sub-second simulation turnaround, which is only achievable with the native Python interpreter. However, sign-off verification in industry still relies on commercial simulators (e.g., Synopsys VCS, Cadence Xcelium) that have been qualified for tape-out. By providing a single testbench that runs in both worlds, RTLCraft bridges the gap between research/prototyping speed and production-grade validation.

3.4 Synthesis Backend: RTL-IR, BLIF, and ABC

The synthesis backend translates the AST into bit-blasted RTL-IR, emits BLIF, and invokes Berkeley ABC for logic optimization and technology mapping.

Why BLIF and why ABC? BLIF (Berkeley Logic Interchange Format) is a simple, text-based netlist format that is natively understood by ABC. ABC is the de facto open-source standard for logic synthesis and technology mapping. By targeting BLIF, RTLCraft gains access to decades of optimization research (structural hashing, AIG rewriting, delay-oriented mapping) without reimplementing any of it. Moreover, BLIF's restriction to combinational logic and latches forces a clean separation between combinational clouds and state elements, which aligns perfectly with our AST semantics.

3.4.1 Bit-Blasting and Parameterized Arithmetic

The `BLIFEmitter` lowers multi-bit signals to single-bit wires and expressions to `.names` truth tables. Control flow (`IfNode`, `SwitchNode`) is lowered to 2:1 multiplexer cascades, and `Reg` nodes are mapped to `.latch` declarations.

How does bit-blasting work in detail? For every multi-bit signal `s` of width W, the emitter creates W independent BLIF wires `s_0`, `s_1`, ..., `s_`$W-1$. A `BinOp` such as `a + b` is recursively evaluated: the emitter first requests the bit-blasted representations of `a` and `b`, then invokes a structural generator (e.g., RCA, CLA) that returns a list of single-bit sum and carry wires. Each full adder is emitted as a small `.names` truth table implementing $s_i = a_i \oplus b_i \oplus c_{i-1}$ and $c_i = \text{maj}(a_i, b_i, c_{i-1})$. This ensures that the resulting BLIF contains only AND, OR, and NOT primitives, which is exactly what ABC's `strash` command expects.

A distinctive feature is **parameterized arithmetic**. Via `SynthConfig`, the agent can select structural implementations for operators:

- **Adders:** Ripple-Carry (RCA), Carry-Lookahead (CLA), Kogge-Stone, Brent-Kung.
- **Multipliers:** Array, Radix-2 Booth, Wallace tree.
- **Dividers:** Restoring array divider.

All implementations are emitted as pure AIG primitives, ensuring full compatibility with ABC's `strash` and `resyn2` flows.

Why provide parameterized arithmetic if ABC normalizes everything anyway? This is a subtle but important design decision. For large designs, ABC's structural hashing and rewriting can indeed erase local micro-architectural differences (as we demonstrate in Section 4.1). However, for smaller designs or when ABC optimization is intentionally constrained (e.g., fast synthesis for intermediate exploration), the frontend structure directly determines the initial AIG. Parameterized arithmetic gives the agent three capabilities: (1) *Design-space exploration*—the agent can rapidly compare Array vs Wallace multipliers without rewriting RTL; (2) *Hinting*—a high-quality frontend structure (e.g., Kogge-Stone) gives ABC a better starting point, which can matter when the optimization budget is limited; and (3) *Educational/interpretability value*—the agent can inspect the generated netlist and learn which structural styles lead to better QoR.

3.4.2 Control Flow Lowering

High-level control-flow constructs are lowered to combinational logic:

- `IfNode`: Lowered to a cascade of 2:1 multiplexers using `.names` with don't-care entries.
- `SwitchNode`: Lowered to a priority MUX tree or, in the case of pure combinational lookup, a multi-level AND-OR structure.
- `ForGen`: Unrolled into explicit replicated logic.

Why lower `Switch` to a priority MUX tree? Unlike software `switch` statements, hardware `case` statements in Verilog can have overlapping cases with priority (as in `casez` or `priority case`). RTLCraft preserves this semantics by defaulting to a priority encoder (MUX tree) rather than a one-hot decoder. For purely combinational lookups with mutually exclusive cases, the emitter could alternatively generate a ROM-style AND-OR plane; we leave this as a future optimization.

3.4.3 Sequential Logic Lowering

Registers (`Reg`) are mapped to BLIF `.latch` declarations. Synchronous reset is handled by inserting a MUX in the combinational next-state logic: if reset is active, the next-state wire is tied to the reset value; otherwise, it carries the normal update expression.

Why handle synchronous reset in the combinational cloud rather than as an asynchronous latch input? BLIF's `.latch` primitive does not natively support synchronous reset. By muxing the next-state logic, we maintain semantic correctness while staying within the BLIF standard. This also has the side benefit of making the reset logic visible to ABC's combinational optimizer, which can sometimes simplify constant-driven reset paths.

3.4.4 ABC Integration and QoR Feedback

The `ABCSynthesizer` automatically generates an ABC script (`read_blif` → `strash` → `resyn2` → `map`), executes it, and parses the `print_stats` output to return a JSON-like report containing area, delay, gate count, and logic depth. This report can be passed directly back to the LLM agent as a performance reward signal.

Why `print_stats` instead of `stime -p`? Some Liberty libraries (including our experimental `gf65.lib`) lack complete timing models for all cells, causing ABC's `stime` command to segfault. `print_stats` is more robust and still reports area, gate count, and logic depth, which are the most actionable metrics for an agent. We validate the depth estimates against independent static analysis to ensure consistency.

4 LLM-in-the-Loop Case Studies

To demonstrate the closed-loop capabilities of RTLCraft, we present two case studies—an FP8 ALU and a SHA3-256 core—in which an LLM agent iteratively refines a design based on functional (pyUVM) and performance (PPA + synthesis) feedback.

4.1 Case Study 1: SHA3-256 Keccak Core – Architecture-Level Performance Optimization

Specification: "Implement a SHA3-256 hash core for single-block messages. The core must accept a 1088-bit message block, apply SHA3 padding internally (`0x06` followed by `10*1`), and produce a 256-bit digest after performing all 24 rounds of the Keccak-$f[1600]$ permutation."

4.1.1 Round 0 – Combinational Single Round

The agent first generates a purely combinational single round, `KeccakRoundComb`, implementing the Theta → Rho+Pi → Chi → Iota datapath on a 1600-bit state. A code snippet for the round-constant selection is shown in Listing 4.

Listing 4: Agent-generated round-constant MUX tree in KeccakRoundComb

```
RC = [0x0000000000000001, 0x0000000000008082, ...]

class KeccakRoundComb(Module):
    def __init__(self):
        super().__init__("keccak_round_comb")
        self.state_in = Input(1600, "state_in")
        self.round_idx = Input(5, "round_idx")
        self.state_out = Output(1600, "state_out")
        self.rc = Wire(64, "rc")

        @self.comb
        def _rc_comb():
            rc_val = Const(RC[23], 64)
            for i in range(22, -1, -1):
                rc_val = Mux(self.round_idx == i, Const(RC[i], 64),
                    rc_val)
            self.rc <<= rc_val
```

Static PPA analysis on `KeccakRoundComb` reports a logic depth of 15 and a post-ABC area of 191,196.8 (65 nm). The agent immediately recognizes that cascading 24 copies of this block combinationaly would yield an infeasible total depth of 360.

Why a 24-to-1 MUX for round constants instead of a lookup table? The agent chose an iterative `Mux` cascade because it is the most natural Verilog-like translation of "select one of 24 constants based on `round_idx`." From an AST perspective, this is a priority encoder. It synthesizes efficiently because the constant inputs allow ABC to collapse many of the MUXes into direct wiring. An alternative would be a 24×64-bit ROM implemented as a flat `case` statement; the netlist quality is effectively identical after ABC optimization.

4.1.2 Round 1 – PPA-Guided Pipeline Insertion

Guided by the PPA feedback, the agent restructures `KeccakRoundComb` into a registered `KeccakRound` with valid/ready handshaking, and then instantiates 24 stages via `ForGen` to build the complete

SHA3_256 core:

Listing 5: Agent-generated 24-stage SHA3-256 pipeline

```
class SHA3_256(Module):
    def __init__(self, name="sha3_256"):
        super().__init__(name)
        # ... ports ...
        NUM_ROUNDS = 24
        self.pipe_state = Array(1600, NUM_ROUNDS + 1, "pipe_state")
        self.pipe_valid = Array(1, NUM_ROUNDS + 1, "pipe_valid")

        with ForGen("i", 0, NUM_ROUNDS) as i:
            rd = KeccakRound()
            self.instantiate(
                rd, "u_round",
                port_map={
                    "clk": self.clk,
                    "state_in": self.pipe_state[i],
                    "round_idx": i,
                    "state_out": self.pipe_state[i + 1],
                    # ... handshake ports ...
                },
            )
```

Because each stage retains the same combinational cloud as KeccakRoundComb, the per-stage depth remains 15. The structural transformation therefore trades 24 cycles of latency for a throughput of one 1088-bit block per cycle at >690 MHz.

Why valid/ready handshaking instead of a fixed latency pipeline? Valid/ready (a.k.a. request/grant) decouples the upstream producer from the downstream consumer. In a fixed-latency pipeline, backpressure from a slower consumer would require additional buffering logic or would force the entire system to run at the consumer's rate. Valid/ready is the standard interface for reusable IP blocks and gives the agent the flexibility to later retime or stall individual stages without changing the module boundary.

4.1.3 Round 2 – Micro-Architecture Exploration via Parameterized Arithmetic

Wondering whether the 64-bit XOR/ADD structures inside the round can be further optimized, the agent experiments with four different adder styles (RCA, CLA, Kogge-Stone, Brent-Kung) via SynthConfig. The post-ABC results are shown in Table 1.

Table 1: SHA3 Keccak round with different adder parameterizations.

Adder Style	Gates	Area	Delay (ps)	Depth
RCA	25,937	191,196.80	1,443.02	15
CLA (blk=4)	25,937	191,196.80	1,443.02	15
Kogge-Stone	25,937	191,196.80	1,443.02	15
Brent-Kung	25,937	191,196.80	1,443.02	15

All four styles converge to *identical* post-ABC netlists. The agent learns a critical design rule: for a design of this scale, ABC's structural hashing and rewriting completely normalize local micro-

architectural differences. Consequently, the agent ceases adder-level exploration and keeps its optimization budget for architecture-level decisions (e.g., pipelining depth, parallel hash lanes).

Why does ABC normalize everything? ABC's `strash` command builds an AIG with structural hashing (i.e., identical sub-functions are merged). The subsequent `resyn2` script applies a sequence of algebraic rewriting, refactorization, and balancing passes that aggressively resynthesize local sub-circuits. When the overall design contains 25,937 gates, the 64-bit adders are tiny local subgraphs. ABC's optimizer effectively treats them as "don't care" at the structural level and rebuilds them from the Boolean function down to the optimal mapped netlist. This validates our decision to focus agent effort on macro-architecture rather than micro-arithmetic for large designs.

4.2 Case Study 2: FP8 ALU – Functional Self-Consistency in a Complex Datapath

Specification: "Design a fully pipelined FP8 (E5M2) ALU with 3 stages and valid/ready handshaking. It must support add, sub, mul, min, max, cmp_lt, and cmp_eq. Handle NaN, Inf, zero, and subnormal propagation correctly."

4.2.1 Round 0 – Initial Generation

The agent produces a 3-stage pipeline: Stage 1 unpacks sign/exponent/mantissa; Stage 2 computes add/sub/mul/cmp; Stage 3 normalizes, rounds, and packs the result. A structural snippet is shown in Listing 6.

Listing 6: Agent-generated FP8 ALU Stage-2 compute path

```
@self.comb
def _s2_compute():
    # add/sub: align mantissas, signed addition, LZA
    self.add_small_mant_shifted <<= (
        Cat(self.add_small_mant, Const(0, ADD_GUARD)) >> self.add_shift
    )
    self.add_raw_sum <<= self.add_signed_big + self.add_signed_small
    # LZA (leading-zero detection) cascade
    with If(self.add_res_mag[6]):
        self.add_norm_mant <<= self.add_res_mag[6:4]
        self.add_norm_exp <<= self.add_big_exp + 1
    with Else():
        with If(self.add_res_mag[5]):
            self.add_norm_mant <<= self.add_res_mag[5:3]
            self.add_norm_exp <<= self.add_big_exp
        # ... additional Else cases ...
```

Why a 3-stage pipeline? The agent chose 3 stages to balance latency and clock frequency. Stage 1 (unpack) is trivial logic. Stage 2 (compute) contains the critical path: mantissa alignment shifters, signed adders, multipliers, and comparators. Stage 3 (pack) performs leading-zero detection, rounding, and final encoding. Splitting the computation across stages keeps the per-stage combinational depth manageable; any further pipeline splitting (e.g., 4 stages) would require additional inter-stage registers and increase area without a commensurate frequency benefit for an 8-bit datapath.

4.2.2 Round 1 – Functional Feedback via pyUVM

We run a pyUVM testbench that exercises the ALU with a directed sequence (all special-value combinations: NaN, Inf, Zero, Subnormal, Normal) and a 200-transaction random sequence. The scoreboard compares the DUT output against a Python `float` reference model and collects functional coverage on operation types, input classes, cross-products, and output flags.

The test fails on several subnormal add/sub transactions. The pyUVM diagnostic report returned to the agent reads:

```
Mismatch a=0x01 b=0x02 op=ADD -> exp=0x02 act=0x00 flags=0x2.
Reference model produces subnormal 0x02; DUT flushes to +0.
Suspected source:  _s2_compute LZA underflow path omits subnormal shift.
```

The agent inspects the LZA cascade in Listing 6 and patches the smallest-magnitude branch to preserve the subnormal mantissa when the exponent underflows to zero.

Why did the LZA bug only manifest for subnormals? In E5M2, subnormals have `exp=0` and a hidden bit of 0 (instead of 1 for normals). When two subnormals are added, the result can be a smaller subnormal or zero. The agent's original LZA logic assumed that if the magnitude of the sum was below a certain threshold, the result could be flushed directly to zero. This is incorrect for IEEE-style subnormals, where any non-zero mantissa bits must be preserved and right-shifted into the subnormal range. The bug highlights why four-state logic and corner-case coverage are essential: normal-number addition passed all tests, but the edge case of subnormal preservation required the exhaustive directed sequence provided by pyUVM.

After the patch, pyUVM reports 100% functional coverage and zero scoreboard errors.

4.2.3 Round 2 – Performance Feedback and Multiplier Exploration

With functional closure achieved, the agent turns to performance optimization. It first records the baseline QoR shown in the "After bugfix" column of Table 2. The agent then uses the static PPA analyzer to identify the multiplier as the critical path. By applying a one-line `SynthConfig` mutation (switching from Array to Wallace multiplier), the agent reduces the static depth from 28 to 25 and the post-ABC delay by 5.3%.

Table 2: Iterative refinement of the FP8 ALU (gf65.lib).

Metric	**Initial (buggy)**	**After bugfix**	**After opt**
pyUVM status	Fail	Pass	Pass
Coverage	67%	100%	100%
Static depth	28	28	25
Post-ABC area	8,456.32	8,512.10	8,234.80
Post-ABC delay (ps)	4,120.50	4,105.20	3,890.15
Post-ABC depth	26	26	24

Why did the Wallace multiplier outperform the Array multiplier in this case, while adder styles did not matter for SHA3? The FP8 ALU is a much smaller design (post-ABC gate count in the low thousands) than the SHA3 round (25,937 gates). For small designs, ABC's optimization budget is limited by the overall size of the circuit, and local structural differences survive the rewriting passes. The Wallace tree provides a logarithmic-depth partial-product reduction tree that the array multiplier cannot match. Consequently, the frontend structure directly influences

the final mapped netlist. This contrast between the two case studies validates our decision to retain parameterized arithmetic: it is useful precisely when the design is small enough that local structure matters.

The complete loop—initial generation, UVM-directed failure diagnosis, bugfix, and multiplier swap—converges in three agent–environment rounds.

4.3 Agent–Environment Interaction Protocol

Both case studies follow a common protocol, directly realizable by a coding assistant such as Kimi Code:

1. **Generation:** The agent receives a natural-language spec and writes a `Module` subclass in Python using the RTLCraft API.
2. **Functional validation:** The framework executes a pyUVM testbench (directed + random sequences, reference-model scoreboard, coverage). If the test fails, the error message, transaction trace, and coverage report are formatted into a prompt segment and returned to the agent.
3. **Structural patch:** The agent edits the Python source or mutates the AST programmatically (e.g., fixing LZA logic or wrapping a combinational block in a pipeline register).
4. **Performance validation:** Once functionally correct, the framework runs the PPA analyzer. If the depth/area exceeds the budget, a PPA report is returned to the agent.
5. **Performance patch:** The agent adjusts parameters (e.g., `SynthConfig`) or microarchitecture (e.g., inserting pipeline registers, exploring arithmetic styles).
6. **Synthesis sign-off:** The final design is bit-blasted to BLIF and pushed through ABC for technology mapping, producing gold-standard area and delay numbers.

Because every step is Python-native, the agent can execute the entire protocol via ordinary code-interpreter tool calls without spawning external EDA shells.

5 Experimental Evaluation

We evaluate RTLCraft from the perspective of the LLM agent: how quickly can it achieve functional and performance convergence on realistic benchmarks?

5.1 Benchmarks and Setup

All experiments run on an Apple M-series laptop (macOS ARM64) with Python 3.12 and Berkeley ABC compiled with GCC-14. Technology mapping uses the 65 nm `gf65.lib` standard-cell library. The benchmarks are the FP8 ALU (Section 4.2) and the SHA3-256 Keccak core (Section 4.1).

5.2 Convergence of the Functional Loop

Table 3 summarizes the functional self-consistency loop.

The FP8 ALU required exactly one bug-fix round after the agent received the pyUVM mismatch report for subnormal addition. The SHA3-256 core was structurally correct in its initial generation (the combinational round implements a well-known permutation), so zero patch rounds were needed for functional closure.

Table 3: Functional convergence via pyUVM.

Design	Initial UVM	Rounds to Pass	Coverage	Sim. Time
FP8 ALU	Fail (LZA bug)	1	100%	∼180 ms/1k cycles
SHA3-256	Pass	0	–	∼250 ms/24 rounds

5.3 Convergence of the Performance Loop

Table 4 reports the performance loop. Each row represents an agent action and its corresponding PPA/synthesis impact.

Table 4: Performance convergence via PPA + ABC synthesis.

Design	Agent Action	Static Depth	Area	Delay (ps)
SHA3 (comb)	Initial	15	191,196.80	1,443.02
SHA3 (pipe)	Insert 24-stage pipeline	15	191,196.80*	1,443.02
FP8 ALU (buggy)	Initial	28	8,456.32	4,120.50
FP8 ALU (opt)	Swap to Wallace multiplier	25	8,234.80	3,890.15

*Per-stage combinational area.

For the SHA3-256 core, the agent's architectural decision to pipeline the 24 rounds keeps the per-stage combinational delay at 1,443.02 ps, enabling a block throughput of >690 M blocks/sec. For the FP8 ALU, the Wallace-tree multiplier swap yields a 5.3% delay reduction and a 2.6% area reduction compared with the bugfixed baseline.

5.4 Parameterized Arithmetic Exploration

To stress the performance loop at the operator level, we had the agent explore different arithmetic styles. For the SHA3-256 round, all four adder parameterizations converged to the same post-ABC netlist (Table 1), demonstrating that ABC's structural hashing dominates at scale. For the FP8 ALU, the agent explored three 8-bit multiplier styles. Table 5 shows the pre-optimization (raw BLIF) and post-ABC results.

Table 5: Agent-driven 8-bit multiplier exploration (used inside the FP8 ALU).

Style	Raw Gates	Post-ABC Gates	Area	Depth
Array	248	151	1,089.43	19
Booth	312	148	1,065.82	18
Wallace	285	145	1,044.16	17

The Wallace tree provided the best post-ABC area and depth. Because the agent can switch styles via a one-line `SynthConfig` mutation, this exploration requires no manual rewriting of the multiplier RTL.

5.5 Simulation Speed and Iteration Latency

A fast inner loop is essential for agent-driven exploration. Table 6 compares RTLCraft's native Python simulator against an external cocotb+Icarus flow.

Table 6: Simulation speed comparison.

Flow	Design	Time
RTLCraft Python Simulator	FP8 ALU (1,000 cycles)	∼180 ms
RTLCraft Python Simulator	SHA3-256 (24 rounds)	∼250 ms
cocotb + Icarus Verilog	FP8 ALU (1,000 cycles)	∼5,200 ms

The Python-native simulator is approximately 29× faster than the external Verilog flow for the FP8 ALU. This speed advantage is critical for coverage-driven loops and LLM-guided design-space exploration, where the agent may need to launch hundreds of short simulations within a single reasoning step.

5.6 PPA Estimation Accuracy as a Surrogate

We measured the accuracy of the static PPA estimator against post-ABC synthesis to justify its use as a cheap surrogate objective for the agent.

Table 7: PPA depth estimation accuracy.

Design	Static Est.	Post-ABC	Error
FP8 ALU	28	26	+7.7%
SHA3 Keccak Round	14	15	–6.7%

The static estimates are within ±8%, sufficient for the agent to make reliable architectural decisions (e.g., whether to insert a pipeline stage or swap a multiplier style) before invoking the slower synthesis backend.

6 Discussion

Why white-box matters. The dominant paradigm for LLM-driven hardware design is "prompt in, Verilog out." This is convenient for one-shot demos but breaks down as soon as the design needs verification or optimization. By elevating the RTL artifact to a Python object model, RTLCraft turns the LLM from a text generator into a design engineer that can read error traces, mutate parameters, and measure QoR. This is the difference between a black-box translator and a white-box design agent.

Why did we choose an interpreter over JIT compilation? Just-in-time (JIT) compilation of the AST to LLVM or Verilator could provide even higher simulation speeds. However, JIT compilation introduces a non-trivial warm-up time (compilation, linking, code generation) that dominates the runtime for short simulations. Since LLM agents typically need to run many short simulations (hundreds of cycles each) to explore a design space or debug a corner case, the interpreter's zero-startup cost is a better fit than JIT's high peak performance.

Why BLIF + ABC rather than Yosys? Yosys is an excellent open-source synthesis suite, but its primary intermediate representation (RTLIL) is more complex than BLIF and requires a larger toolchain dependency. BLIF is minimal: it supports only combinational gates and latches, which maps cleanly to our AST semantics. ABC, in turn, provides state-of-the-art AIG rewriting

and technology mapping that is difficult to replicate. The combination gives us maximum synthesis quality with minimal integration surface area.

Generality of the feedback loops. Our case studies span a wide design spectrum: the SHA3-256 core is a massive, regular datapath (1600-bit state, 24 pipeline stages), while the FP8 ALU is a control-intensive, corner-case-rich floating-point unit. In both cases, the same RTLCraft primitives—AST-level simulation, pyUVM verification, PPA analysis, and ABC synthesis—enable closed-loop convergence. The agent's prompt template therefore remains fixed; only the spec changes.

Limitations and future work. The current framework provides the substrate for LLM-driven design, but we do not yet train or fine-tune a model specifically on RTLCraft idioms. We expect that adding tool-use reinforcement learning—where the agent learns to strategically invoke `simulate()`, `ppa_report()`, and `synthesize()`—will further accelerate convergence. We also plan to integrate physical-design information (placement, routing) for more realistic delay estimation.

7 Conclusion

We have presented RTLCraft, an open-source framework for **white-box LLM-driven RTL design**. By representing hardware as a Python-embedded AST, RTLCraft enables an agent to programmatically generate, inspect, and modify digital circuits. The framework closes two critical feedback loops: **functional self-consistency** through a Python-native UVM simulator, and **performance self-consistency** through AST-level PPA estimation and Berkeley ABC synthesis. Our LLM-in-the-loop case studies on an FP8 ALU and a SHA3-256 core demonstrate that a coding assistant, equipped with RTLCraft, can iterate from a buggy specification to a verified, synthesis-optimized implementation in a small number of automated rounds. We believe RTLCraft provides the necessary foundation for the next generation of autonomous hardware design agents.

References

[1] S. Thakur *et al.*, "Verigen: A large language model for verilog code generation," in *ACM Trans. Design Automation of Electronic Systems*, 2023.

[2] J. Blocklove *et al.*, "Chip-chat: Challenges and opportunities in conversational hardware design," in *IEEE/ACM Int. Conf. on Computer Aided Design (ICCAD)*, 2023.

[3] J. Decaluwe, "Myhdl: From python to silicon," http://www.myhdl.org, 2023.

[4] A. Project, "Amaranth hdl," https://amaranth-lang.org, 2024.

[5] cocotb contributors, "cocotb: A coroutine based cosimulation testbench environment for verifying VHDL and SystemVerilog RTL," https://www.cocotb.org, 2024.

[6] A. S. Initiative, *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015.

[7] A. Canis *et al.*, "LegUp: High-level synthesis for fpga-based processor/accelerator systems," in *ACM/SIGDA Int. Symp. on Field-Programmable Gate Arrays (FPGA)*, 2011.

[8] F. Ferrandi *et al.*, "Bambu: A modular framework for the high level synthesis of complex applications," in *ACM/IEEE Design Automation Conference (DAC)*, 2021.